

eAPTS System Phase 1 Code Audit

Comprehensive technical audit of the Electronic Auditable Pharmaceutical Transactions and Services platform. This report identifies 46 findings across authentication, data integrity, API security, and compliance readiness, with detailed remediation strategies for production deployment.

Classification: Confidential

Document Type: Security Assessment Report

Prepared For: Chief Technology Officer

23 CRITICAL

14 HIGH

9 MEDIUM

Table of Contents

1. Executive Summary	2
2. System Architecture Overview	3
2.1 Django Application Structure	3
2.2 Database Schema Assessment	3
3. Critical Findings: Authentication and Data Integrity	4
3.1 Credential and Configuration Exposure	4
3.2 Authentication Architecture Failures	5
3.3 Data Integrity and Audit Trail Deficiencies	6
3.4 API Security Gaps	7
4. High Findings: Security and Business Logic	8
5. Medium Findings: Performance and Code Quality	10
6. Frontend Architecture Assessment	11
7. Compliance Readiness Assessment	12
7.1 Good Distribution Practice (GDP) Alignment	12
7.2 21 CFR Part 11 (Electronic Records and Signatures)	12
7.3 National Pharmacy Board Requirements	13
8. Prioritized Remediation Roadmap	14

1. Executive Summary

This report presents the findings of a comprehensive technical audit conducted on the Electronic Auditable Pharmaceutical Transactions and Services (eAPTS) system, a national-level pharmaceutical supply chain management platform. The audit was performed as Phase 1 of a multi-phase transformation initiative, intended to elevate the system from its current prototype state to a production-ready, regulatorily compliant platform suitable for deployment across hospital environments nationwide.

The eAPTS system is designed to digitize the pharmaceutical supply chain by tracking medicine stock, managing sales and dispensing operations, and creating an immutable audit trail to prevent fraud, theft, and medical errors. Given its deployment in hospital settings where data accuracy directly impacts patient safety, the system must meet the highest standards of data integrity, auditability, and security. The regulatory framework governing such systems includes Good Distribution Practice (GDP), 21 CFR Part 11 (electronic records and signatures), and national pharmacy board compliance requirements.

The audit scope covered the entire technology stack: the Django REST Framework (DRF) backend, comprising three Django applications (inventory, locations, person), the React.js frontend built with Create React App (CRA) and Material UI (MUI), the database schema (currently SQLite, with PostgreSQL planned), the authentication and authorization subsystem, and the overall architectural patterns. A total of **46 findings** were identified, classified into three severity tiers as summarized below.

Severity	Count	Categories	Risk Level
CRITICAL	23	Authentication, Data Integrity, Security, Architecture	Immediate action required
HIGH	14	Security, Business Logic, Compliance	Address within 2-4 weeks
MEDIUM	9	Performance, Code Quality, Operations	Address within 1-2 sprints

Table 1. Finding Severity Distribution

The most urgent concerns center on the absence of a unified authentication system, the lack of any audit trail infrastructure, unprotected API endpoints, and the use of SQLite in what is intended to be a national-scale deployment. These findings represent fundamental architectural gaps that must be resolved before any feature development can proceed safely. The following sections detail each finding category with specific remediation strategies grounded in Django and DRF best practices.

2. System Architecture Overview

The eAPTS backend is structured as a Django 4.2.24 project with three distinct Django applications, each responsible for a specific domain boundary. The frontend is a React 19 application using Material UI (MUI) v7 and React Router v7 for navigation. The following table summarizes the current technology stack and its assessed maturity level.

Component	Technology	Version	Maturity Assessment
Backend Framework	Django + DRF	4.2.24 / 3.16.1	Framework configured; application logic incomplete
Frontend Framework	React + MUI	19.1.1 / 7.3.2	Routing defined; auth and API integration missing
Database	SQLite (planned PostgreSQL)	3.x	CRITICAL: Not production-viable for multi-pharmacy use
Authentication	Django sessions (JWT planned)	SimpleJWT 5.5.1	CRITICAL: JWT installed but unconfigured; dual auth conflict
API Documentation	drf-yasg (installed)	1.21.10	Installed but not wired into any URL configuration
Password Hashing	bcrypt (installed)	4.3.0	Installed but unused; make_password defaults to PBKDF2
Build Tool (Frontend)	Create React App	5.0.1	EOL/deprecated; should migrate to Vite

Table 2. Technology Stack Maturity Assessment

2.1 Django Application Structure

The backend is organized into three Django apps under the `apps/` namespace: `apps.inventory` (drug catalog, stock management, dispensing, prescriptions, stock transfers), `apps.locations` (pharmacy locations, location hierarchies, location tags), and `apps.person` (persons, patients, custom users, roles, privileges). The domain modeling shown in the UML class diagram demonstrates thoughtful separation of concerns, with proper use of foreign key relationships and a role-based access control (RBAC) schema. However, the application layer beneath these models is severely underdeveloped, as detailed in the findings below.

2.2 Database Schema Assessment

The database comprises 23 business tables across the three applications, plus 10 Django system tables. The schema was migrated via 21 migration files. Notably, the current deployment uses SQLite as the

database engine, with all tables and migrations confirmed present. The schema design correctly employs BigAutoField primary keys, appropriate foreign key relationships with DEFERRABLE INITIALLY DEFERRED constraints, and unique constraints on key lookup fields (e.g., `medical_record_number`, `username`). However, several critical constraints are missing, which are catalogued in the findings below.

3. Critical Findings: Authentication and Data Integrity

This section presents the 23 critical-severity findings, organized into four thematic subsections: credential and configuration exposure, authentication architecture failures, data integrity and audit trail deficiencies, and API security gaps. Each finding represents a risk that could result in unauthorized data access, data loss, regulatory non-compliance, or patient safety incidents if left unaddressed.

3.1 Credential and Configuration Exposure

The current codebase exhibits multiple instances of sensitive credential exposure and configuration mismanagement. In a pharmaceutical system that handles controlled substances and protected health information, these vulnerabilities represent an immediate threat to data confidentiality and system integrity. The following table summarizes the credential-related findings.

ID	Finding	Description
#1	Secrets in .env likely committed to VCS	Database password, Django secret key, and DEBUG flag stored in a .env file without gitignore verification
#2	.env file never loaded (load_dotenv missing)	python-dotenv installed but never called; all config hardcoded in settings.py with different values than .env
#9	SQLite database shipped with repository	db.sqlite3 (416 KB) containing session data and user records present in upload directory
#10	ALLOWED_HOSTS empty	Empty list will cause 400 errors on all requests when DEBUG=False in production
#11	drf-yasg installed but not configured	API documentation generator in requirements but absent from INSTALLED_APPS and URL configuration

Table 3. Credential and Configuration Exposure Findings

Remediation Strategy for Credential Management:

Immediately rotate all exposed credentials: the Django secret key, the PostgreSQL password, and any session keys. Ensure that `.env` is listed in `.gitignore` before any further commits. Refactor `settings.py` to use `django-environ` or `python-dotenv` with a proper `load_dotenv()` call at the top of the settings module. Adopt a split-settings pattern (e.g., `settings/base.py`, `settings/development.py`, `settings/production.py`) to separate environment-specific configuration. For production, read secrets from environment variables or a secure vault service rather than from files. Configure `ALLOWED_HOSTS` from an environment variable, and set `DEBUG=False` in production settings with no possibility of accidental override.

3.2 Authentication Architecture Failures

The authentication system is the most severely compromised subsystem in the eAPTS platform. Multiple coexisting authentication mechanisms, none of which are properly configured, create a fragmented and exploitable security posture. The core issue is a fundamental architectural conflict between Django's built-in authentication and a custom user model that was intended to support RBAC but was never properly integrated.

ID	Finding	Description
#5	No AUTH_USER_MODEL configured	Custom person.User model exists but Django still uses auth_user as authoritative source
#31	authenticate() checks wrong model	django.contrib.auth.authenticate validates against auth_user, not person_user
#32	Fragile password hashing in User.save()	Only checks pbkdf2_ prefix; bcrypt installed but unused; no set_password/check_password
#33	Session auth in DRF API (architectural mismatch)	Uses Django sessions instead of JWT; stateful, non-scalable, incompatible with SPA
#34	Login endpoint leaks user enumeration	Returns 400 (not 401); no rate limiting; no account lockout; no failed-login logging
#35	No user-location association validation	Any authenticated user can select any location without authorization check
#36	Login response omits role/permission data	Returns only location_tag; frontend has no data for RBAC decisions
#37	RBAC system completely non-functional	Role, Privilege, UserRole, RolePrivilege tables exist but are never queried
#38	Duplicate login_api function	Two versions exist (api_views.py and views.py); dead code imports from locations app
#39	person_user password stored but never validated	Custom model hashes password but authenticate() only checks auth_user.password

Table 4. Authentication Architecture Failures

Remediation Strategy for Authentication:

The single most impactful remediation is to unify the authentication system. The recommended approach follows Django best practices and involves several coordinated changes. First, set `AUTH_USER_MODEL = "person.User"` in settings and refactor the custom User model to extend `AbstractBaseUser` with proper `USERNAME_FIELD` and `PASSWORD_HASHERS` configuration. Implement `check_password()` and `set_password()` methods on the User model, delegating to Django's built-in hashing framework which will automatically use `bcrypt` if configured in `settings.PASSWORD_HASHERS`. Remove the custom `save()` override that manually checks for the `pbkdf2_` prefix, as this is fragile and bypasses the framework's password management.

Second, configure `SimpleJWT` properly in `settings.py` with appropriate token lifetimes, and add JWT authentication to the DRF default authentication classes. Create a custom login endpoint that authenticates against the custom User model, issues JWT access and refresh tokens, and returns the user's roles and privileges in the response payload. Third, build a `HasRolePermission` custom DRF permission class that queries the Role and RolePrivilege tables to enforce access control on every API endpoint. Wire this permission class into the DRF global default permissions and override it on specific views as needed. Fourth, add rate limiting with `rest_framework.throttling.AnonRateThrottle` on the login endpoint (e.g., 5 attempts per minute), implement account lockout after 10 failed attempts, and log all authentication events (successful and failed) to an audit log table.

3.3 Data Integrity and Audit Trail Deficiencies

In a pharmaceutical system, data integrity is not merely a technical requirement but a regulatory obligation. Every state change in the system, from stock receipt to dispensing, must be traceable to a responsible actor with an immutable timestamp. The current codebase lacks the fundamental infrastructure to support this requirement. There are no audit tables, no timestamp fields on business models, and no mechanism to prevent silent data modification or deletion.

ID	Finding	Description
#12	inventory/views.py is completely empty	Zero API endpoints for drugs, stock, dispensing, prescriptions, or transfers
#13	No model-level constraints on Stock	No <code>unique_together</code> on (drug, location, batch); no <code>quantity >= 0</code> check; no expiry validation
#14	No timestamps on any model	Missing <code>created_at</code> , <code>updated_at</code> , <code>created_by</code> on all 23 business tables
#15	No soft delete mechanism	Hard delete via <code>CASCADE</code> on all models; no <code>is_deleted</code> or <code>deleted_at</code> fields
#16	Prescription and Dispensing are orphaned	No FK linking dispensing to prescription; dispensing possible without prescription

ID	Finding	Description
#17	Stock decrement has no integrity protection	No select_for_update, no atomic transactions, no stock movement log
#18	Zero test coverage (inventory)	tests.py empty in all three Django apps; no model, view, or integration tests
#23	Hard delete on all DestroyAPIView	DELETE /api/locations/{id}/ permanently destroys data; cascades wipe related records
#24	Tag update uses delete-all-reinsert	LocationTagMap update destroys history, non-atomic, no transaction wrapper

Table 5. Data Integrity and Audit Trail Deficiencies

Remediation Strategy for Data Integrity:

Create a `TimeStampedModel` abstract base class that adds `created_at`, `updated_at` (both with `auto_now_add` and `auto_now` respectively), and `created_by` (a `ForeignKey` to the `User` model). All business models should inherit from this base class. Additionally, create a `SoftDeleteModel` mixin that overrides `delete()` to set `is_deleted=True` and `deleted_at=now()` instead of removing the record, and adds a default manager that filters out soft-deleted records.

Implement an `AuditLog` model that records every create, update, and delete operation on business models using Django signals or a custom middleware. Each audit entry should capture: the acting user, the model and record ID, the field-level changes (old value and new value), the timestamp, the IP address, and the action type. For stock operations specifically, create a `StockMovement` model that records every quantity change with the reason (received, dispensed, transferred, adjusted, expired), the batch number, the previous and new quantity, and the responsible user. Wrap all stock-modifying operations in `transaction.atomic()` blocks with `select_for_update()` to prevent race conditions.

Add database-level constraints using `unique_together` on `Stock` for (`drug`, `location`, `batch_number`), add a `CheckConstraint` for `quantity >= 0`, and add a `ForeignKey` from `Dispensing` to `Prescription` to enforce prescription-dispensing linkage. Replace all `DestroyAPIView` with custom views that perform soft deletes, and configure Django admin to prevent hard deletes via `has_delete_permission` overrides.

3.4 API Security Gaps

The REST API surface of the eAPTS system is entirely unprotected. With no authentication, authorization, rate limiting, or input validation configured at the framework level, every endpoint is accessible to any anonymous user on the network. In a hospital environment where the API handles patient records, controlled substance inventory, and dispensing data, this represents an unacceptable security posture.

ID	Finding	Description
#3	SimpleJWT installed but never configured	No REST_FRAMEWORK settings; no JWT auth classes; no token lifetimes
#4	No DRF configuration at all	No DEFAULT_AUTHENTICATION_CLASSES, DEFAULT_PERMISSION_CLASSES, pagination, or throttling
#6	Inventory API has no URL routes	The most critical domain (drugs, stock, dispensing) is completely disconnected from the API
#22	All location endpoints unprotected	ListCreate and RetrieveUpdateDestroy views have zero permission_classes

Table 6. API Security Gap Findings

Remediation Strategy for API Security:

Configure the DRF global settings in `settings.py` with JWT as the default authentication class, `IsAuthenticated` as the default permission class, global pagination with `PageNumberPagination` (page size 50), and rate limiting using `UserRateThrottle` (e.g., 100 requests per minute) and `AnonRateThrottle` (e.g., 10 requests per minute). Add `django-filter` as the default filter backend. Register the inventory URLs in the root URL configuration. Create serializers and viewsets for all inventory models (Drug, Stock, Dispensing, Prescription, StockTransferRequest) with appropriate permission classes that enforce role-based access control. Add input validation in serializers using DRF field validators and custom `validate_` methods.

4. High Findings: Security and Business Logic

The high-severity findings represent significant vulnerabilities and design deficiencies that, while not posing an immediate threat to system availability, create substantial risk in a production deployment. These findings span production security hardening, business logic gaps, and structural issues in the authentication and frontend layers. Left unaddressed, these issues would result in a system that functions but fails to meet the security and compliance standards required for a national pharmaceutical platform.

ID	Finding	Description
#7	Zero production security headers	Missing <code>SECURE_SSL_REDIRECT</code> , <code>SESSION_COOKIE_SECURE</code> , <code>HSTS</code> , <code>X_FRAME_OPTIONS</code> , <code>SECURE_CONTENT_TYPE_NOSNIFF</code>
#8	No logging configuration	Zero <code>LOGGING</code> setup; no audit log file paths, structured format, or log rotation

ID	Finding	Description
#16	Prescription-Dispensing orphaned	No FK linking dispensing to prescription; unlinked dispensing is a compliance gap
#17	Stock decrement integrity	No select_for_update, atomic transactions, or stock movement logging
#18	Zero test coverage (all apps)	tests.py empty in inventory, locations, and person apps
#24	Tag update delete-all-reinsert pattern	Destroys tag change history; non-atomic; no transaction wrapper
#25	N+1 query risk (no select_related)	Location listing triggers O(N) queries for parent hierarchy and tags
#40	User.status has no choices validation	Free-text field with no enforced values (active, inactive, suspended)
#41	No serializers for person models	User, Patient, Role, Privilege cannot be managed via API
#42	CSRF endpoint not a proper API view	Returns JsonResponse instead of DRF Response; inconsistent with rest of API
#43	Cross-app import violations	person app imports locations.models directly, creating tight coupling

Table 7. High-Severity Findings

Remediation Strategy for High-Severity Findings:

For production security headers, add the following to the production settings: `SECURE_SSL_REDIRECT = True`, `SESSION_COOKIE_SECURE = True`, `CSRF_COOKIE_SECURE = True`, `SECURE_HSTS_SECONDS = 31536000`, `SECURE_HSTS_INCLUDE_SUBDOMAINS = True`, `SECURE_CONTENT_TYPE_NOSNIFF = True`, and `X_FRAME_OPTIONS = "DENY"`. Deploy behind a reverse proxy (nginx) that terminates TLS and sets `SECURE_PROXY_SSL_HEADER`.

For logging, configure Django's `LOGGING` dict with separate handlers for security events (authentication attempts, permission denials), inventory operations (stock changes, dispensing, transfers), and system events. Use a JSON formatter for machine-parseable logs, implement log rotation via `RotatingFileHandler`, and send critical logs to a centralized log aggregation service in production. For the N+1 query issue, add `select_related("parent")` and `prefetch_related("tags__location_tag")` to the Location queryset, and remove the `write_only` flag from the tags serializer field or create a separate read serializer. For test coverage, establish a testing foundation with `pytest` and `pytest-django`, prioritizing tests for authentication flows, stock operations, and dispensing logic, targeting a minimum of 80% code coverage before production deployment.

5. Medium Findings: Performance and Code Quality

Medium-severity findings address performance bottlenecks, code maintainability concerns, and operational gaps that affect the long-term health and scalability of the platform. While these issues do not pose immediate security risks, they represent technical debt that will compound over time and hinder the system's ability to scale to a national deployment across hundreds of pharmacy locations.

ID	Finding	Description
#9	Database config inconsistency	settings.py says PostgreSQL; db.sqlite3 exists; psycopg2 commented out in requirements
#11	drf-yasg not wired	API documentation tool installed but absent from INSTALLED_APPS and URL conf
#19	Bare admin registrations	All 13 models registered with zero customization (no filters, search, readonly fields)
#20	DrugAttributeType.data_type untyped	Free-text field with no validation against expected values (string, integer, date)
#21	DispensingItem stores price separately	Drug.price and DispensingItem.price can diverge; no price-at-dispensing snapshot
#25	No select_related on querysets	N+1 queries on location parent hierarchy and tag lookups
#26	Location.name has no unique constraint	Multiple locations can share same name; identification is ambiguous
#27	Location.description missing from model	Present in UML but not in Django model; inconsistent with LocationTag
#28	No pagination on any endpoint	Large list responses will cause slow loads and memory pressure at scale
#29	No filtering or search	django-filter installed but unused; no search, filter, or ordering capabilities
#30	tests.py empty (locations app)	Third consecutive empty test file; zero coverage on the only app with endpoints
#44	Patient PHI unprotected in API	No field-level encryption, data masking, or access logging for health information
#45	Person.__str__ returns only ID and gender	Admin dropdowns show "42 - M" instead of human-readable names
#46	tests.py empty (person app)	All three Django apps have zero test coverage

Table 8. Medium-Severity Findings

Remediation Strategy for Medium-Severity Findings:

Resolve the database inconsistency by uncommenting `psycpg2-binary` in requirements, verifying PostgreSQL is running, and removing the SQLite file from the repository. For the Django admin, create custom `ModelAdmin` classes for each business model with `list_display`, `list_filter`, `search_fields`, `readonly_fields` for audit columns, and `has_delete_permission` returning `False` to prevent hard deletes. Configure global pagination with `PageNumberPagination` (page size 50) and add `django-filter` as the default filter backend with appropriate `filterset_class` definitions for each view. Wire `drf-yasg` into the URL configuration to provide auto-generated Swagger/OpenAPI documentation. Fix the `Person.__str__` method to traverse the `PersonName` relationship and display the full name with a fallback to the ID when no name exists.

6. Frontend Architecture Assessment

While the primary audit scope is the backend, the frontend codebase exhibits several critical issues that directly impact the security posture of the overall system. The React application uses Create React App (CRA) which is in maintenance mode and should be migrated to Vite for long-term support. Material UI v7 is used for the component library, and React Router v7 handles client-side routing. Axios is installed for HTTP requests but has no configured instance, interceptors, or error handling layer.

Finding	Description
Authentication state in React useState	User state lost on refresh; no token persistence; routes unprotected
No client-side route guards	Admin, store, dispensing, and cashier routes render regardless of auth state
No RBAC in router	No role checking; any user can navigate to admin pages like <code>/admin/users</code>
No axios instance configured	No base URL, interceptors, auth token attachment, or error handling
CRA is EOL/deprecated	Should migrate to Vite for build performance and long-term support
Boilerplate leftovers	<code>App.css</code> has spinning logo; <code>manifest.json</code> says "Create React App Sample"
Missing production dependencies	No state management (Redux/Zustand), form validation (react-hook-form), or error boundaries

Table 9. Frontend Architecture Findings

Remediation Strategy for Frontend:

Implement JWT token persistence using `localStorage` (access token) and an in-memory store or `httpOnly` cookie (refresh token). Create a React context or use Zustand for global auth state management that persists across page refreshes. Implement route guards as higher-order components or

layout-level checks that verify the JWT token and the user's role before rendering protected routes. Configure an Axios instance with a base URL, request interceptor (attaching the JWT Bearer token), and response interceptor (handling 401 errors with automatic token refresh). Migrate from CRA to Vite for improved build performance and active maintenance. Clean up boilerplate files (remove spinning logo, update manifest.json, customize README).

7. Compliance Readiness Assessment

A national-level pharmaceutical transaction system must comply with multiple regulatory frameworks that govern electronic records, data integrity, and audit trails. This section maps the current findings to specific regulatory requirements and describes how the recommended remediations bring the system toward compliance. The three primary regulatory frameworks considered are Good Distribution Practice (GDP), 21 CFR Part 11 (FDA electronic records and signatures), and the national pharmacy board's requirements for pharmaceutical information systems.

7.1 Good Distribution Practice (GDP) Alignment

GDP requires complete traceability of pharmaceutical products throughout the supply chain, from receipt at the warehouse to dispensing to the patient. Every movement of stock must be documented with date, time, responsible person, and reason. The current system cannot satisfy this requirement because there are no stock movement records, no timestamp fields on business models, and no dispensing-to-prescription linkage. The recommended `StockMovement` model and `AuditLog` infrastructure directly address this gap by creating an immutable record of every quantity change with full attribution. The addition of `created_at`, `updated_at`, and `created_by` timestamps on all models ensures that every record can be placed in a temporal sequence with a responsible actor.

7.2 21 CFR Part 11 (Electronic Records and Signatures)

21 CFR Part 11 establishes criteria under which electronic records and electronic signatures are considered trustworthy, reliable, and equivalent to paper records. Key requirements include: audit trails that capture who, what, when, and why for every record modification; system-generated time stamps that cannot be altered; operational system checks to enforce permitted sequencing of steps; and authority checks to ensure only authorized individuals can access the system or perform critical operations. The current system fails on all four counts. The recommended remediations address each requirement: the `AuditLog` model provides the "who, what, when, why" trail; the `TimeStampedModel` provides system-generated timestamps; the stock integrity controls (atomic transactions with `select_for_update`) enforce correct sequencing; and the RBAC permission system with JWT authentication ensures only authorized individuals can perform operations. The soft-delete mechanism ensures that records are never truly removed, only superseded, maintaining the complete record history

required by Part 11.

7.3 National Pharmacy Board Requirements

National pharmacy boards typically require that pharmaceutical information systems maintain accurate, complete, and contemporaneous records; restrict access to authorized personnel only; provide a mechanism for data backup and recovery; and implement controls to prevent unauthorized modification of records. The migration from SQLite to PostgreSQL addresses the backup and recovery requirement by enabling point-in-time recovery, WAL-based replication, and automated backup tools such as `pg_dump`. The RBAC system with JWT authentication addresses the access restriction requirement. The immutable audit log addresses the record modification control requirement. The production security headers (HSTS, secure cookies, content type sniffing protection) address data-in-transit security requirements.

Requirement	Current Status	Post-Remediation Status	Key Changes
Full audit trail (who/what/when/why)	Not implemented	Compliant	AuditLog model, TimeStampedModel
Electronic record integrity	Not enforced	Compliant	Soft delete, atomic transactions
User authentication and authorization	Broken (dual auth)	Compliant	Unified auth, JWT, RBAC permissions
Access control (role-based)	Schema exists, non-functional	Compliant	HasRolePermission class, RBAC enforcement
Stock traceability	No movement records	Compliant	StockMovement model, batch tracking
Prescription-dispensing linkage	No FK relationship	Compliant	FK from Dispensing to Prescription
Data backup and recovery	SQLite (single-file)	Compliant	PostgreSQL with WAL and PITR
Production security (HTTPS, headers)	Not configured	Compliant	Security headers, TLS, reverse proxy
API documentation	Installed but not wired	Compliant	drf-yasg properly configured
Test coverage	0%	Target: 80%+	pytest, pytest-django, CI pipeline

Table 10. Compliance Readiness Matrix

8. Prioritized Remediation Roadmap

The following roadmap organizes the remediation work into three phases, ordered by priority and dependency. Phase 1 addresses the most critical architectural gaps that must be resolved before any feature work can proceed. Phase 2 addresses security hardening and business logic gaps. Phase 3 addresses performance optimization, code quality, and operational readiness. Each phase is designed to be deliverable within a defined timeframe with measurable milestones.

Phase	Timeline	Key Deliverables	Findings Addressed
Phase 1: Foundation	Weeks 1-4	Unified auth (JWT + RBAC), AuditLog model, TimeStampedModel, PostgreSQL migration, DRF config, inventory URLs	#1-6, #12-15, #22-23, #31-39
Phase 2: Hardening	Weeks 5-8	StockMovement model, prescription linkage, soft delete, security headers, logging, test foundation, rate limiting	#7-8, #16-18, #24-25, #40-44
Phase 3: Polish	Weeks 9-12	Admin customization, pagination/filtering, drf-yasg docs, frontend auth, CRA-to-Vite migration, 80% test coverage	#9-11, #19-21, #26-30, #45-46

Table 11. Prioritized Remediation Roadmap

It is strongly recommended that no feature development proceed until Phase 1 is complete. The foundations laid in Phase 1, particularly the unified authentication system, audit infrastructure, and database migration, are prerequisites for all subsequent work. Attempting to build features on the current architecture would result in code that must be substantially rewritten once the foundational changes are in place, wasting development effort and introducing additional risk.

Following the completion of all three phases, the system should undergo a formal compliance audit against the regulatory frameworks described in Section 7 before being cleared for production deployment. This audit should be performed by an independent third party with expertise in pharmaceutical information systems and electronic record compliance.